

CCAR-P Domain 3: Session Summary

Everything covered in this chat on the kaveri-dock-lab repository: the eight lab scripts run and explained, the book pages read, the One Ticket walkthrough, the Margin arithmetic drill, and the Wrong Tower red-pen exercise. Written so it stands alone as revision material before Dock Book II.

1. What was done to the repository

Change	Why
Moved sop_214.md into sops/	e4_chunking.py and e1_kaveri_mcp.py both open sops/sop_214.md. Merged as PR #1.
Added .gitignore	Python bytecode cache (__pycache__ /) appeared once e5 imported e4.
Added e5_hybrid_explained.pdf	Two-page explainer for the hybrid search file.
Added e9_one_ticket.py	The eight hops of E1-E8 on one request, in the house style.
Installed fastmcp, reportlab, pypdf	Needed to run e1 and to build PDFs. Not committed; reinstall in a fresh container.

Standing rule agreed mid-session: only the standalone .py files in the repository are executed, never code transcribed from the Loading Dock book.

2. The eight scripts: how to run, what they print, what they teach

Script	Run	Key output	Lesson (objective)
e1_kaveri_mcp.py	python e1_probe.py (server alone only waits for a client)	2 tools, 1 resource sop://214, 1 prompt; schema generated from the type hints; call returns CUSTOMS_HOLD	Tools act, resources inform, prompts template. One server, many clients. (3.7)
e2_bloat_audit.py	DRY_RUN=1 python e2_bloat_audit.py	9 tools 716 tok, 3 tools 265 tok; belt tax 18.04M tokens/day (live count_tokens needs an API key)	Idle schemas x per-schema x daily calls. Trim by logs. (3.1)
e3_gate_pass.py	python e3_gate_pass.py	kavi-agent DENY, refund-desk ALLOW, carrier-portal-v1 ALLOW via wildcard; one-line fix makes it DENY	authorize, then audit_log, then tool. Log denials. Narrow wildcards. (3.2)
e4_chunking.py	python e4_chunking.py	fixed: 8 chunks, procedure split across boxes 4 and 5; heading: 10 chunks, procedure whole	Chunk along meaning. Overlap mends sentences, not sections. (3.5)
e5_hybrid.py	DRY_RUN=1 python e5_hybrid.py	'e-way bill re-filing' -> section 4; 'parcel stuck, paperwork problem' -> section 1	Keyword register + synonym foreman, normalised, blended 60/40. (3.5/3.6)
e6_shape_router.py	python e6_shape_router.py (sets DRY_RUN itself)	KF-id -> structured_lookup; 'how many' -> sql_aggregate; else -> hybrid_rag citing section 4	Retrieval follows the shape of the data. Rules in order. (3.6)
e7_dock_cameras.py	python e7_dock_cameras.py	one ticket, 4 JSON lines, one trace id; digest flags retrieve: mean 395 ms, p95 2,349 ms	Trace id minted once. Report p95, not mean. (3.4)
e8_two_bills.py	python e8_two_bills.py	monolithic 362,460; progressive 4,205 + 1 round-trip; small+cached 4,404; gap 86x	Map first, territory on demand. Small+stable+always-used: cache instead. (3.8)

3. The deep dives, in one paragraph each

E4 chunking

Three cutters on SOP 214. The fixed cutter counts 180 words and cuts, so it tore Step 2 in half and left Step 1 in box 4 and Step 6 in box 5. The heading cutter splits before every # or ## line and glues a label [SOP 214 - title] on top, so section 4 stays whole at 236 words. with_overlap carries the last 30 words of each box forward: it mended the torn sentences but still did not pull Step 1 across. Overlap is a bandage for cut sentences, not a cure for cut sections.

E5 hybrid search

The register (bm25ish) is word counting with two fixes: rare words count more (re-filing in 5 of 10 chunks weighs 0.69, bill in 7 of 10 weighs 0.38) and repeats saturate ($\text{count}/(\text{count}+1.2)$), so 12 mentions barely beat 9). The foreman (toy_embed) drops hyphenated and numeric words, swaps synonyms (parcel to consignment, stuck to hold), and compares bags by cosine. norm() divides each judge by its max so both top out at 1.0; hybrid() blends 0.6 register + 0.4 foreman. On the jargon query the foreman preferred section 3 but the register's 60% carried section 4; on the everyday query both agreed on section 1.

E7 cameras

log() prints one JSON line per event. one_ticket() mints the trace id outside the loop so all four stage lines share it; minted inside, the ticket is shredded. p95 sorts and reads position $0.95n-1$, so one 9,000 ms outlier among nineteen 30 ms tickets moves the mean to 478 and the p95 not at all. digest() piles 10,000 tickets by stage and flags retrieve: 94% finish near 250 ms, 6% near 2,631 ms, and the mean of 395 describes nobody.

E8 two bills

Monolithic stuffs 400 SOPs (360,000 of 362,460 tokens; the question is 0.03%). Progressive sends 9 one-line stubs, a 260-token index and 20 chunks: 4,205 tokens plus one round-trip, 86x cheaper. The honest third case: 45 SOPs cached at 10% lands at 4,404 with no extra turn. Retrieval is for material too big or too changeable to carry.

4. Book pages read with the photos

Context editing, server-side

Old advice: summarise old turns yourself. Current: header context-management-2025-06-27 with strategy clear_tool_uses_20250919 (default trigger 100k input tokens, keep 3 recent pairs, exclude_tools to protect names) and clear_thinking_20251015. Clearing is deletion, not compaction. Every clear invalidates the cached prefix from that point, so clear_at_least makes you clear in one big block and pay the rewrite once. Save essentials with the memory_20250818 tool before clearing. Verified against the current API reference.

Cardinality, symptom alerting, the five-layer stack

Categories (stage, carrier, route) belong in metrics; ids (trace, consignment) belong in logs and traces, or the metrics store melts under 40,000 new series a day. Alert on symptoms customers feel, few and dear. Law 4: logs say what happened, traces say where, metrics say whether it is normal. The incident walk goes down the stack in one minute: alert at 10:41, dashboard names retrieve, metric says since 10:15, trace shows the 2.8 s carrier call, log line names KF-2481 and BlueDart. Traps: dashboard theater, and 'the mean is inside budget'.

5. One Ticket, End to End

Sunita Kulkarni types 'Where is KF-2481 right now, and what do I do to get it out of customs hold?' at 10:41. 2.6 seconds and 47 paise later she has an answer citing SOP 214 section 4 that promises no date. Every objective fires once, in order.

Hop	What happens	ms	Obj	Repo
1 Door	SSO identity attached; trace id t-7f3a minted before anything is answered. Both identities travel: caller kavi-agent, user sunita.k	0	3.4	e7
2 Router	Id pattern -> structured lookup; procedure prose -> hybrid RAG. Neither half goes to the wrong machine	94	3.6	e6
3 Gate	authorize(kavi-agent, get_shipment_status) needs status:read; audit line written before the tool body	6	3.2	e3
4 Connector	tools/call returns CUSTOMS_HOLD (140 ms); resources/read sop://214 costs no tool call (30 ms); customs_triage prompt fetched at session start	170	3.7	e1
5 Shelves	BM25 + vector, 150 each, fused, reranked (+198 ms), top 20. Top box is section 4, whole, because chunks are heading-cut and self-labelled	520	3.5	e4, e5
6 Belt + window	8,000-token preamble cached (~800 billed), 4-tool belt 1,040, 20 chunks 3,600, question 140. About 5,800 vs 362,480 monolithic	20	3.1, 3.8	e2, e8
7 Dials	Sonnet-class (draft error ~4 rupees), k=20, rerank on, BlueDart ETA with 3 s timeout ran concurrently in 610 ms	1,760	3.3	none
8 Cameras	Nine JSON lines, one trace id. Answer cites SOP 214 v7 S4, promised_date false	30	3.4	e7

Remove one hop

Take each hop away and ask what breaks and when you would find out. Only the stuffed corpus fails immediately. Every other removal works on Tuesday and breaks later: no trace id means the slow ticket cannot be explained; no scope check means the wildcard reads every hold at 03:00; both intents to RAG means a plausible wrong hold reason; fixed chunks means the cancellation answer; nine tools means 1,500 extra tokens on 40,000 calls; no timeout means a 40 s hang; free-text logs means no p95 and no denial spike. The demo lacked all of them and worked.

How the five laws hold each other up

Law	Only possible because	Feeds
1 Every tool is a door (3.1, 3.2)	the cameras logged which tools were called (3.4)	the belt that rides in the window (3.8)
2 Chunk along meaning (3.5)	the corpus was big enough to need RAG	what the dials can buy at query time (3.3, 3.6)
3 Retrieval follows shape (3.6)	a cheap known-category router sits in front	which machine, so which bill, each intent pays
4 You cannot fix what you cannot see (3.4)	a trace id at the door, every stage timed alone	every number in Parts 2, 3, 6, 8
5 Map first, territory on demand (3.8)	the connector can list before it calls (3.7); retrieval replaces stuffing (3.5)	a window with room to think

The exam mixes objectives in one stem because no objective stands alone. The correct option usually respects the hop before the one the stem names.

e9_one_ticket.py

Written this session in the E-file house style: a header per hop naming the E file it borrows from. It reuses the real E1 server through the in-process client, E4's by_heading and E5's hybrid, and writes nine register lines under one trace id. One lesson came from building it: sending the whole question to the shelves retrieved section 1, because the toy foreman maps 'customs' to 'hold'. Letting hop 4's fact (missing: e-way bill) sharpen hop 5's query fixed it. Hops interlock; break the chain and the answer goes wrong.

6. The Margin: eight arithmetic shapes

Recognise the shape, compute in ten seconds, spend the stem on the tell. A stem with three numbers wants exactly two combined. All 24 variants were worked and matched the sealed key.

#	Pattern (obj)	Method	Variants worked	Tell
1	Belt tax (3.1)	idle x tokens/schema x calls/day	42M; 108M; 9.6M saved	Two count_tokens readings: subtract them
2	Blended cost vs ceiling (3.3)	sum(share x price) vs ceiling	2.33; 1.45; 9.40 breaches by 3.40	'All routes' means share 1.00
3	p95 + addition vs SLA (3.3/3.4)	p95 + added, compared at p95	2.05 > 2 breach; 3.35 < 4 fits but refuse k=40; 4.1 > 3	The mean is a decoy, never in the sum
4	Skip-RAG threshold (3.5/3.8)	pages x 400 vs 200k; stable; always used	38k cache; 1.4M RAG; 150k but 30% -> retrieve	Three conditions, not one
5	Cache-read equivalents (3.8)	cached x 0.1; prefix >= tier minimum	1,376; 6,000 vs 3,600 cache still wins; 2,000 < 4,096 never caches	Token column alone misleads
6	Batch feasibility (3.8, D1)	fits a window? then latency x cases vs window	72 s fits; 13 min fits; 1.8M vs 18M uncached	Window first, then time
7	Overlap ratio (3.5)	overlap / chunk; index ~ 1 + ratio	80% too far; 11% seam insurance; 1.35M indexed	Overlap near chunk size stores everything twice
8	Cardinality (3.4)	multiply distinct label values	144 fine; 160,000/day trap; 16,000/day still trap	Sampling an id does not make it a category

In one breath: belt tax and blended cost are linear, sanity-check magnitude. Latency adds at p95. ~400 tokens a page, ~200k is the skip-RAG line with stable and always-used alongside. Cache reads ~0.1x, minimum prefix 1,024-4,096 by tier. Feasibility before taste. Overlap/chunk = duplicated fraction. Labels multiply, ids explode.

7. The Wrong Tower: thirty flaws in a fluent proposal

A vendor proposal with 30 planted flaws and 6 decoys that look wrong and are right. Target: 24+ found, 2 or fewer false alarms. The flaws by objective, with the most important fix in each cluster:

Obj	Flaws	The cluster's lesson
3.7 connector	5	SOPs are resources not tools; a procedure is a prompt; a negotiating agent is not a stateless tool; 'SSE' and 'sampling' are stale vocabulary; do not wrap a single-consumer endpoint 'for consistency'
3.1 belt	2	Ten idle tools = 104M tokens/day; identical descriptions on two tools raise the wrong-tool rate
3.2 gate	5	One shared identity kills attribution; prompt rules are not permission checks; log denials; rotating a wildcard does not narrow it; least privilege applies to resources
3.5 shelves	6	Fixed chunks, vector-only, labels outside the chunk, top-5, monthly rebuild of a weekly corpus, rerank refused everywhere and situating misplaced at query time
3.6 shape	3	Ids want lookup not similarity; RAG cannot count; exact codes are the keyword register's game
3.3 dials	3	1.9 + 0.2 breaches 2.0; sequential carrier calls with no timeout; one mid-tier model for both 1,800-rupee and 4-rupee errors
3.4 cameras	3	Free-text logs, a metric per consignment id, health as the mean; alert on everything; dashboard shipped with the launch
3.8 context	2	Full corpus 'offset by caching' fits no window; progressive fetch of a 45k always-used pack; 2,000-token prefix below Haiku's 4,096 minimum
scale trap	1	A 12-dispatcher pilot promoted as-is

The six decoys: one MCP server for four clients; escalate_to_human kept at two calls; SSO for people and API keys for services; the 90-day rotation itself; a single embedding model; marking the preamble for caching. Each sits beside a real flaw in the same paragraph. The most dangerous sentences were the calm ones: 'to simplify operations', 'for consistency', 'as best practice recommends', 'well within the SLA'.

8. Reference cards

Distractor families

Code	Family	What the wrong option does
OE	over-engineered	adds complexity that does not help; 'fixes' a correct decision
VG	vocabulary gap	uses the right word for the wrong thing (mean for tail, SSE for Streamable HTTP)
TR	trade-off reversed	pays the cost without the benefit
CM	constraint missed	ignores a stated limit (SLA, ceiling, minimum, consequence)
HL	human loop skipped	automates a decision a person should make
OP	over-permissive	grants more than the job needs
SF	silent failure	swallows an error or a denial
SD	sounds diligent	looks thorough, does nothing (dashboard theater, prompt-based permissions)

The eight hops in one breath

- Door: identity in, trace id minted (3.4).
- Router: id to lookup, procedure prose to hybrid RAG (3.6).
- Gate: authorize, audit_log, tool; denials logged (3.2).
- Connector: tools/call for status, resources/read for SOP 214, prompt for the procedure (3.7).
- Shelves: heading chunks, situated, BM25 + vector, fuse ~150, rerank, top 20 (3.5).
- Belt + window: lean belt, cached preamble, retrieved chunks, question (3.1 + 3.8).
- Dials: mid-tier where errors are cheap, rerank where the SLA allows, 3 s timeouts (3.3).
- Cameras: one JSON line per stage with ms, tokens, pause; p95 not means (3.4).

Running everything in one go

```
cd kaveri-dock-lab && pip install fastmcp
for f in e1_probe.py e2_bloat_audit.py e3_gate_pass.py e4_chunking.py e5_hybrid.py \
    e6_shape_router.py e7_dock_cameras.py e8_two_bills.py e9_one_ticket.py; do
    echo "## $f"; DRY_RUN=1 python "$f"; done
```

What is left

The repository also holds Dock Book II (45 fresh questions, ninety minutes, cold), the Dock Walk, the Dock Lab Kit, the Storybook and the main Loading Dock volume. The One Ticket document names Dock Book II as the next sitting. Re-sit the Wrong Tower in a week with the overlay hidden, target 27+ with at most one false alarm.

Every number in this summary was either printed by a repo script in this session or read from the book's own answer keys. Where the two disagree (E2's 716 against the book's 1,740 for nine schemas) the book's figure is the live count_tokens result and the script's is a four-chars-per-token estimate.